Truth Progress Tracker

Document: `progress-tracker.md`
Project: Truth
Purpose: Maintain an accurate, continuously updated record of implementation progress, verification results, blockers, architectural decisions, and the next actionable step.

---

1. Core Rule

The progress tracker is the operational record of Truth's development. Its purpose is to answer five questions at any point in the project:

1. What are we building?
2. What has actually been implemented?
3. What has been tested and verified?
4. What remains incomplete or blocked?
5. What is the exact next step?

No progress may be claimed without supporting evidence. A discussed idea is not an implementation. A written specification is not a working feature. Code that exists is not necessarily tested. A passing test does not prove that every aspect of a feature works.

The tracker MUST reflect the actual repository and available verification evidence, not assumptions, intentions, or previous progress claims that have not been confirmed.

---

2. Tracking Status Definitions

- **PLANNED:** The work has been identified but implementation has not started.
- **IN PROGRESS:** Implementation has started but the task is not complete.
- **IMPLEMENTED:** The relevant code or artifact exists. Required verification may still be incomplete.
- **VERIFIED:** The task's defined acceptance criteria have been checked, and the available evidence supports completion. (For code tasks, this normally requires relevant tests and applicable quality checks. For documentation tasks, it requires reviewing the saved document and confirming that it meets its requirements.)

- **BLOCKED:** Work cannot proceed because a dependency, decision, access requirement, technical problem, or other obstacle must first be resolved. Record the specific blocker and what is needed to remove it.
- **DEFERRED:** Work has deliberately been postponed by an approved decision. Record the reason and any condition for resuming it.
- **SUPERSEDED:** A previous task, design, or implementation has been replaced by an approved alternative. Preserve the reason and relevant history rather than silently erasing the decision.

Do not use VERIFIED merely because code was written, an agent reported success, or a command was attempted.

---

3. Project Identity and Scope

- **Truth** is the active project name.
- The project's current technical foundation is a minimal executable prototype of the Truth Kernel.
- The established conceptual execution flow is: `Input` → `Observation` → `Memory Retrieval` → `State` → `Reasoning` → `Response` → `Commit`
- The long-term direction includes a locally operated, self-contained system where feasible, replaceable AI integrations, controlled application access, personal context, and governed agents.

These future capabilities MUST remain distinct from current implementation status. The tracker must not imply that Truth already has complete local autonomy, a production mobile application, continuous background operation, self-hosted AI, or autonomous company operations unless those capabilities have been implemented and verified.

---

4. Master Progress Register

Maintain one authoritative register of development tasks:

Task ID	Workstream	Task	Status	Dependencies	Acceptance criteria	Verification evidence	Blocker	Next action
DOC-001	Documentation	Save project-overview.md	VERIFIED	None	Document saved and verified	Document file created and inspected	None	None
DOC-002	Documentation	Save architecture.md	VERIFIED	DOC-001	Document saved and verified	Document file created and inspected	None	None
DOC-003	Documentation	Save ai-workflow-rules.md	VERIFIED	DOC-002	Document saved and verified	Document file created and inspected	None	None
DOC-004	Documentation	Save progress-tracker.md	VERIFIED	DOC-003	Document saved and verified	Document file created and inspected	None	None
SYS-001	Repository	Inspect repository and reconcile local vs remote state	PLANNED	DOC-004	Actual repository inspected and technical baseline established	None	Repository checkout needed	Inspect actual files and dependencies

5. Six-File Instruction Set

Track the project's instruction documents independently from software implementation. The initial six documents are:

File	Responsibility	Completion condition	Status
`project-overview.md`	Purpose, vision, scope, and priorities	Saved and reviewed	**VERIFIED**
`architecture.md`	System structure and component boundaries	Saved and reviewed	**VERIFIED**
`ai-workflow-rules.md`	Coding agent workflow and operating rules	Saved and reviewed	**VERIFIED**
`code-standards.md`	Code quality, safety, and testing standards	Saved and reviewed	**PLANNED**
`ui-context.md`	Interface responsibilities and interaction standards	Saved and reviewed	**PLANNED**
`progress-tracker.md`	Development status and continuity	Saved and reviewed	**VERIFIED**

---

6. Current Technical Baseline

- **Known project direction:** A minimal executable Truth Kernel prototype using Python, FastAPI, Uvicorn (`uvicorn app.main:app --reload`), and pytest.
- **Status:** Awaiting repository inspection (`SYS-001`) to verify actual implementation files, dependencies, routes, and test suites.

---

7. Acceptance Criteria for Every Task

Each meaningful task MUST have clear acceptance criteria before it is considered complete. Acceptance criteria should describe observable outcomes rather than vague intentions, validate inputs/outputs, preserve architectural boundaries, and include tests where appropriate.

---

8. Verification Evidence

Every `VERIFIED` task must have evidence appropriate to its type (passing tests, file existence review, verified interaction behavior). Do not claim unexecuted verification.

---

9. Dependency and Blocker Tracking

Tasks must respect dependency order. Blockers must be recorded with concrete reasons and required resolutions.

---

10. Architectural Decision Log

Decision ID	**Date**	**Context / Problem**	**Decision made**	**Alternatives considered**	**Consequences / Trade-offs**	**Status**
ADR-001	2026-10-23	Initial documentation baseline	Save core foundational specs (overview, architecture, workflow rules, progress tracker)	Single monolithic README	Clear separation of concerns and adherence to strict project guidelines	**Approved**

---

11. Change and Regression Tracking

Changes to files, public interfaces, data schemas, and dependencies must be recorded and checked against architectural invariants.

---

12. Session Continuity Protocol

Every development session begins by reconciling the tracker with actual project state, verifying the last milestone, and selecting the next actionable task according to dependencies.

---

13. Progress Reporting Format

Reports must follow the structure: Current milestone, Completed and verified, Implemented but not verified, In progress, Blocked or deferred, Changes made, Verification results, and Next action.

---

14. Prioritization Rules

Priority order: Security/data-integrity risks → Core functionality blockers → Architecture/interface contracts → Core Kernel functionality → Tests/verification → Developer documentation → UI features → Optional optimizations.

---

15. Long-Term Roadmap Versus Current Implementation

Long-term capabilities (local autonomy, self-hosted AI, autonomous agents) remain distinct from current implementation status and are not introduced prematurely.

---

16. Tracker Maintenance Rules

Maintain one authoritative tracker, preserve stable task identifiers, verify repository facts, separate implementation from verification, and keep the next action specific and actionable.

---

17. Initial Status and First Update

The initial documentation baseline (`DOC-001` through `DOC-004`) has been established. Codebase repository inspection (`SYS-001`) is pending.

---

18. Definition of Done for the Tracker

The progress tracker is complete when vocabulary is explicit, tasks have acceptance criteria, implementation and verification are distinguished, and session handoff is defined.